Enterprise DevSecOps Pipeline

4-Week Security Automation Demo

Built around OWASP Juice Shop

Infotact Cybersecurity Project 3

github.com/sidmoharatha-beep/ecommerce-enterprise-devsecops-pipeline

1. Project Overview

DevSecOps means integrating security directly into the development and operations process instead of treating it as a separate step. This project demonstrates a complete, automated DevSecOps pipeline that scans software at multiple stages before it reaches production.

We use OWASP Juice Shop, an intentionally vulnerable online shop application, as our target. This lets us show how each security tool detects real issues.

The pipeline is split across three contributor branches, each representing one week of work, so the project can be understood and presented in clear stages.

2. Repository & Branch Structure

Repository: github.com/sidmoharatha-beep/ecommerce-enterprise-devsecops-pipeline

Branch Distribution

- sidhartha-branch -> Week 1: Application Containerization + SAST
- chayan-branch -> Week 2: Dependency SCA + Container Image Scanning
- aditya-branch -> Week 3: IaC Scanning + Week 4: DAST + Pipeline Hardening

Key Files

- app/ -> OWASP Juice Shop source code
- Dockerfile -> Hardened container image build
- .github/workflows/devsecops-pipeline.yml -> Main CI/CD pipeline
- .github/workflows/iac-scan.yml -> Infrastructure scan workflow
- terraform/ -> Secure AWS deployment definition
- docs/ -> Week-by-week documentation
- sonar-project.properties -> SonarQube scan configuration

3. Week 1 - Containerization + SAST (sidhartha-branch)

Week 1 focuses on building a secure container and scanning the source code for bugs and bad practices.

What was added

- Hardened Dockerfile: uses a non-root user, multi-stage build, minimal Alpine base, dependency pinning, disabled post-install scripts, and a health check.
- OWASP Juice Shop placed under app/ as the intentionally vulnerable target.
- SonarQube SAST integrated into GitHub Actions.
- sonar-project.properties configures SonarQube rules and project key.

Required secrets

Add these in GitHub Settings -> Secrets and variables -> Actions:

- SONAR_TOKEN -> token generated from SonarCloud or self-hosted SonarQube
- SONAR_HOST_URL -> https://sonarcloud.io for SonarCloud, or your self-hosted URL

4. Week 2 - Dependency + Container Scanning (chayan-branch)

Week 2 adds Software Composition Analysis (SCA) and container image scanning using Trivy.

What was added

- Trivy filesystem scan: detects vulnerable dependencies in the project.
- Trivy container image scan: detects CVEs in the built Docker image.
- Scan results uploaded as GitHub Actions artifacts.
- Pipeline fails on HIGH/CRITICAL findings so issues are caught early.

Why it matters

Most modern applications use open-source libraries. If one has a known vulnerability, it becomes your vulnerability. Trivy finds these before deployment.

5. Week 3 - IaC Scanning (aditya-branch)

Week 3 adds Infrastructure as Code (IaC) scanning so cloud misconfigurations are caught before any resources are created.

What was added

- Terraform configuration under terraform/ for a minimal secure AWS deployment.
- Architecture: VPC, public/private subnets, Application Load Balancer, ECS Fargate, encrypted S3 bucket, CloudWatch logs, restrictive security groups.
- iac-scan.yml workflow runs Terraform init, validate, Checkov scan, and TFSec scan.
- SARIF and JSON artifacts are uploaded for review.

Why it matters

A single misconfigured S3 bucket or open security group can expose a whole system. IaC scanners enforce security policies automatically.

6. Week 4 - DAST + Pipeline Hardening (aditya-branch)

Week 4 tests the running application and secures the CI/CD workflow itself.

What was added

- OWASP ZAP baseline scan runs against a locally built Juice Shop container.
- Dependency review runs on every pull request.
- TruffleHog scans commit history for leaked secrets and credentials.
- GitHub TOKEN permissions are restricted to the minimum needed.
- Action versions are pinned for supply-chain security.
- ZAP report is uploaded as an artifact.

Why it matters

SAST and SCA find code-level issues. DAST finds runtime issues that only appear when the application is running. Pipeline hardening protects the CI/CD system itself from abuse.

7. How the Pipeline Works

When code is pushed or a pull request is opened, the following stages execute automatically:

```
Code Push / Pull Request
        |
        v
Secret Scan  (TruffleHog)
        |
        v
Dependency Review  (PR only)
        |
        v
SAST  (SonarQube)
        |
        v
Docker Build
        |
        v
SCA + Image Scan  (Trivy)
        |
        v
DAST  (OWASP ZAP)

Separate workflow:
Terraform Validate -> Checkov -> TFSec
```

Each stage acts as a gate. If a critical issue is found, the pipeline stops, preventing insecure code from moving forward.

8. Why This Project Is Useful

• Defense in depth: security checks run at source, dependency, container, infrastructure, and runtime layers.

• Shift-left security: problems are found early, when they are cheaper and faster to fix.

• Automation: once configured, every push and PR is automatically scanned.

• Real-world tooling: uses industry-standard tools that companies actually use.

• Demonstrable: because Juice Shop is intentionally vulnerable, every tool produces meaningful, easy-to-understand results.

• Collaborative workflow: branches mirror a team dividing work across weeks and contributors.

9. How to Demo

- Open the GitHub repository in a browser.
- Show the three branches: sidhartha-branch, chayan-branch, aditya-branch.
- Open the Actions tab and show a recent workflow run.
- Open .github/workflows/devsecops-pipeline.yml and explain each job.
- Open terraform/main.tf and describe the secure AWS design.
- Show the IaC scan workflow and its artifacts.
- Show the uploaded ZAP report artifact.
- Open Settings -> Secrets and variables -> Actions to show SONAR_TOKEN and SONAR_HOST_URL are configured.
- Explain the difference between SAST, SCA, IaC scanning, and DAST.

10. Quick Summary

This project proves that a multi-layer DevSecOps pipeline can be built entirely with open-source tools and GitHub Actions. It secures code before it becomes a container, secures containers before they run, secures infrastructure before it is deployed, and tests the running application for exploitable weaknesses.

Repository: github.com/sidmoharatha-beep/ecommerce-enterprise-devsecops-pipeline

Main tools: GitHub Actions, Docker, SonarQube, Trivy, Checkov, TFSec, OWASP ZAP, TruffleHog, Terraform